

LogiMeat — Documentación Técnica Profesional

Campo	Valor
Producto	LogiMeat ERP
Organización	Colbeef SAS
Dominio	Planificación y seguimiento logístico de carne (beneficio, desposte, celfrío, subproductos)
Versión documentada	Código fuente del repositorio logimeat (2026)
Audiencia	Ingenieros de software, analistas TIC, mantenedores del sistema
Autoría UI	Daniel Almeida Jaimes · Colbeef SAS

Índice

1. [Resumen ejecutivo](#)
2. [Objetivo del sistema](#)
3. [Stack tecnológico y lenguajes](#)
4. [Arquitectura del software](#)
5. [Estructura del repositorio](#)
6. [Núcleo de infraestructura](#)
7. [Autenticación, sesión y roles](#)
8. [Módulos funcionales](#)
9. [Lógica de negocio: Programación](#)
10. [CRUD genérico de maestros](#)
11. [Módulo de personal](#)
12. [Modelo de datos](#)
13. [Flujos de usuario](#)
14. [Seguridad](#)
15. [Frontend y assets](#)
16. [Configuración y despliegue](#)
17. [Scripts y migraciones](#)
18. [Convenciones para un ingeniero senior](#)
19. [Mapa de archivos por responsabilidad](#)

- 20. [Guía de onboarding](#)
 - 21. [Diagrama de flujo](#)
 - 22. [Glosario](#)
-

1. Resumen ejecutivo

LogiMeat es una aplicación web interna de Colbeef SAS orientada a la **programación operativa logística** de productos cárnicos. No es un WMS ni un POS: su entidad central es la tabla transaccional **Programacion**, que representa viajes/movimientos operativos (kg, cliente, planta, OPL, conductor, vehículo, estado y atributos OTIF).

El sistema también cubre:

- Consulta y análisis (dashboard, estadísticas, OTIF, calendario).
- Catálogos maestros (clientes, OPL, conductores, vehículos, productos, etc.).
- Gestión de personal (empleados, descansos, turnos semanales).
- Observabilidad de uso (tablero de usabilidad para Super Admin).

Se integra al ecosistema intranet **WORKBEEF** mediante enlace fijo a <http://192.168.20.205:8000/site.html>.

2. Objetivo del sistema

Objetivo	Cómo lo cubre LogiMeat
Registrar la demanda operativa	Alta de programaciones con ID de negocio único
Coordinar logística	Asociación OPL ↔ conductor ↔ vehículo
Medir calidad de entrega	Campos OTIF / pedido perfecto + pantallas de calidad
Planificar personal	Tablero semanal de descansos y turnos
Administrar catálogos	Hub maestros.php + CRUD gestion_tabla.php
Controlar acceso	Sesión PHP, roles, timeout 15 min, CSRF

3. Stack tecnológico y lenguajes

3.1 Resumen por capa

Capa	Tecnología	Lenguaje / formato
Backend	PHP 8.x procedural (page controllers)	PHP (<code>declare(strict_types=1)</code> en módulos críticos)
Base de datos	MySQL / MariaDB vía PDO	SQL + DML embebido en PHP
Presentación	HTML5 + Tailwind CSS (compilado local)	HTML, CSS
Interactividad	JavaScript vanilla + librerías UMD	JavaScript
Tipografía	Plus Jakarta Sans (woff2 locales)	Assets estáticos
Correo	SMTP propio o <code>mail()</code>	PHP sockets / SMTP
Diagramas / docs	Markdown + draw.io	Markdown, XML draw.io

3.2 Lenguajes utilizados (detalle)

PHP (lenguaje principal del servidor)

- Versión objetivo: **PHP 8.x** (uso de `match`, `str_starts_with`, tipado estricto, `Throwable`).
- Estilo: **procedural por página** (no MVC formal, no Composer, no ORM).
- Persistencia: **PDO** con `ERRMODE_EXCEPTION`, `FETCH_ASSOC`, prepares reales (`EMULATE_PREPARES = false`).
- Sesiones: `$_SESSION` con timeout de inactividad.
- Prefijo de helpers: `lm_*` (LogiMeat) y funciones de dominio (`programacion_*`, etc.).

SQL (MySQL)

- Charset: `utf8mb4`.
- Base por defecto: `db_logimeat`.

- Esquema legado + migraciones parciales en `config/*.sql` y `scripts/aplicar_schema_*.php`.
- Particularidad crítica: `Fecha_de_Operacion` en `Programacion` se almacena como **texto d/m/Y**, no como tipo `DATE`.

HTML / CSS

- Plantillas embebidas en cada `.php` (sin motor de plantillas).
- Estilos con **Tailwind CSS 3.x** precompilado en `assets/vendor/tailwind-built.css` (diseño **offline-first**, sin CDN).

JavaScript

- Scripts inline o en página para modales CRUD, filtros de formularios y calendarios.
- **Chart.js** (`chart.umd.min.js`) en dashboard/estadísticas.
- **FullCalendar** (`fullcalendar.index.global.min.js`) en vista calendario.

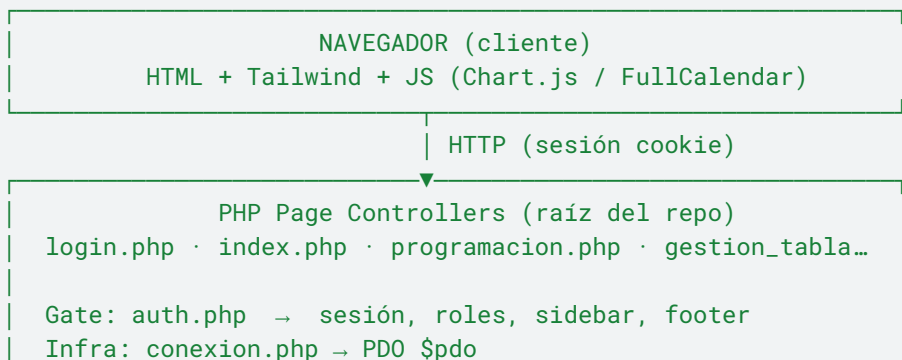
3.3 Lo que no usa el proyecto

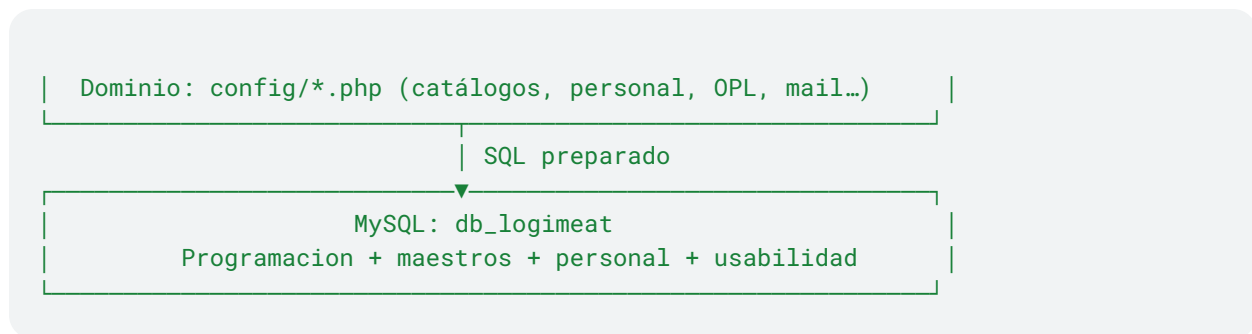
- No hay `composer.json` / dependencias PHP Composer.
- No hay framework (Laravel, Symfony, etc.).
- No hay ORM (Eloquent, Doctrine).
- No hay SPA (React/Vue/Angular).
- No hay librerías PDF/Excel en el código.
- No hay API REST pública documentada (salvo endpoints JSON puntuales como `eventos_calendario.php`).

4. Arquitectura del software

4.1 Patrón arquitectónico

None





Patrón: *Page Controller* procedural. Cada archivo PHP de la raíz es una pantalla (o endpoint) autocontenida:

1. Incluye `auth.php` (si es área privada).
2. Incluye `conexion.php` (PDO).
3. Ejecuta lógica SQL / validaciones.
4. Renderiza HTML + JS.

La base de datos actúa como modelo. No existe capa de servicios formal ni repositorios.

4.2 Capas lógicas

Capa	Ubicación	Responsabilidad
Presentación	<code>*.php</code> en la raíz	UI, formularios, listados, dashboards
Dominio compartido	<code>config/programacion_*.php</code> , <code>personal_helpers.php</code> , <code>logisticos_vinculo_maestro.php</code>	Reglas de catálogo, OPL, disponibilidad personal
Infraestructura	<code>conexion.php</code> , <code>config/seguridad.php</code> , <code>mail_send.php</code> , <code>lm_assets.php</code>	BD, headers, CSRF, correo, assets
Mantenimiento	<code>scripts/*.php</code> , <code>config/*.sql</code>	Migraciones, imports CLI

5. Estructura del repositorio

None

```
logimeat/
├─ index.php, login.php, auth.php, conexion.php
├─ programacion.php, nueva_programacion.php, editar_programacion.php
├─ procesar_programacion.php
├─ gestion_tabla.php, maestros.php
├─ view_data.php, eventos_calendario.php, logistica.php
├─ estadisticas.php, otif.php
├─ tablero_descansos.php, personal*_form.php, empleados_importar.php
├─ password_reset*.php, cambiar_password.php
├─ tablero_usabilidad.php, tablero_usabilidad_datos.php
├─ conexion.local.example.php      # plantilla (versionada)
├─ conexion.local.php              # credenciales (NO versionar)
├─ assets/
│   └─ fonts/                     # Plus Jakarta Sans
│   └─ vendor/                    # Tailwind, Chart.js, FullCalendar
│   └─ colbeef-logo.svg
├─ config/                        # helpers + schemas SQL parciales
├─ scripts/                       # migraciones / import CLI
├─ DOCUMENTACION.md              # este documento
└─ diagrama_flujo_logimeat.drawio # diagrama editable en draw.io
```

No existen carpetas `src/`, `models/`, `controllers/`, `tests/` automatizados ni `vendor/` de Composer.

6. Núcleo de infraestructura

6.1 Conexión a base de datos — `conexion.php`

Jerarquía de configuración (de menor a mayor prioridad efectiva):

1. Defaults de desarrollo (`127.0.0.1`, `db_logimeat`, `root/root`, puerto `3306`).
2. Variables de entorno: `DB_HOST`, `DB_NAME`, `DB_USER`, `DB_PASS`, `DB_PORT`, `DB_CHARSET`.
3. Override local: `conexion.local.php` (array merge; **gitignore**).

Resultado: instancia global `$pdo` (PDO MySQL).

Efecto colateral: si hay sesión autenticada en petición web, registra evento de usabilidad.

6.2 Seguridad transversal — `config/seguridad.php`

- Endurecimiento de cookies de sesión.
- Headers HTTP (`X-Content-Type-Options`, `X-Frame-Options`, etc.).
- Helpers CSRF: `lm_csrf_*`.

6.3 Assets offline — `config/lm_assets.php`

- Resuelve rutas de CSS/JS/fuentes locales.
- Soporta `web_base` / `LM_WEB_BASE` si la app vive en subcarpeta del DocumentRoot.
- Función típica de cabecera: `lm_head_local_assets()`.

6.4 Correo — `config/mail_send.php`

Usado en recuperación de contraseña. SMTP configurable en `conexion.local.php` (host mail Colbeef / Office 365).

7. Autenticación, sesión y roles

7.1 Login — `login.php`

1. Recibe `usuario` + `clave` por POST.
2. Consulta: `SELECT * FROM User WHERE Nombre = ? AND Clave = ?`.
3. Si coincide, establece sesión y redirige a `index.php`.
4. Registra login en usabilidad.

Nota de seguridad (importante para mantenedores): las contraseñas se comparan en **texto plano** (no hay `password_hash` / `password_verify` en el flujo actual). Cualquier endurecimiento futuro debe migrar hashes y actualizar login + reset.

7.2 Gate de sesión — `auth.php`

Incluido al inicio de casi todas las pantallas privadas:

Control	Comportamiento
Sin <code>user_id</code>	Redirect a <code>login.php</code>
Inactividad > 900 s (15 min)	Destruye sesión → <code>login.php?error=sesion_expirada</code>

Control	Comportamiento
Actividad válida	Renueva <code>ultima_actividad</code>

También expone:

- `lm_rol_actual()`, `lm_es_admin()`, `lm_es_super_admin()`, `lm_es_operativo()`
- `mostrarSidebar($activePage)` — menú lateral por rol
- Footer / overlay de carga

7.3 Mapeo de roles (en login)

Condición en BD / nombre	Rol efectivo en sesión
Nombre <code>ANALISTA TIC</code>	Super Admin
Rol <code>ADMIN / ADMINISTRADOR</code>	Administrador
Rol <code>AUXILIAR / OPERATIVO</code>	Operativo
Otro	valor crudo o Operativo

7.4 Matriz de permisos (funcional)

Módulo	Operativo	Administrador	Super Admin
Dashboard / Estadísticas / Programación (alta) / Calendario / Logística / OTIF	✓	✓	✓
Editar programación		✓	✓
Tablero personal		✓	✓
Configuración (maestros)		✓	✓
Gestión tabla <code>user</code>			✓
Tablero usabilidad			✓

7.5 Recuperación de contraseña

1. `password_reset_request.php` — solicita email, genera token (~45 min).
 2. `password_reset_confirm.php?token=...` — actualiza `User.Clave`.
 3. Helpers: `config/password_reset_helpers.php` (tabla `password_reset`, columna `User.Email`).
-

8. Módulos funcionales

8.1 Navegación (sidebar)

1. **Dashboard** → `index.php`
2. **Estadísticas** → `estadisticas.php`
3. **Programación** → `programacion.php`
4. **Calendario** → `view_data.php`
5. **Conductores / Vehículos** → `logistica.php`
6. **Calidad OTIF** → `otif.php`
7. **Tablero personal** → `tablero_descansos.php` (*no operativo*)
8. **Configuración** → `maestros.php` (*admin*)
9. **Usabilidad** → `tablero_usabilidad.php` (*super admin*)

Acciones de pie: cambiar contraseña, volver a WORKBEEF, logout.

8.2 Dashboard — `index.php`

KPIs agregados sobre `Programacion` (kg, viajes, OTIF, cumplimiento) + gráficos Chart.js. Hay corte temporal de datos en lógica de negocio (referencia desde `2026-04-30` en código).

8.3 Estadísticas — `estadisticas.php`

Vistas por periodo: planta, clientes, actividad, producto, cobertura logística.

8.4 Calendario — `view_data.php` + `eventos_calendario.php`

- UI FullCalendar.
- Endpoint JSON de eventos: `eventos_calendario.php`.

Observación de seguridad: `eventos_calendario.php` históricamente puede no pasar por `auth.php`. En revisiones futuras conviene protegerlo igual que el resto de pantallas privadas.

8.5 Logística — `logistica.php`

Catálogo / cruces históricos de conductores y vehículos a partir de `Programacion`.

8.6 OTIF — `otif.php`

Vista de calidad OTIF. Parte de las métricas se calculan desde BD (`Estado_Actividad`); otras pueden estar parcialmente fijadas en UI (puntualidad/completitud). Los campos de pedido perfecto existen en el formulario de programación.

8.7 Usabilidad — `tablero_usabilidad.php`

Dashboard de uso (logins / páginas visitadas) restringido a Super Admin. Persistencia en `app_usabilidad_evento` vía `config/usabilidad_log.php`.

9. Lógica de negocio: Programación

La **programación** es el corazón del sistema.

9.1 Archivos del flujo

Archivo	Rol
<code>programacion.php</code>	Listado filtrable; agrupación por planta; acciones Nueva / Editar / Duplicar
<code>nueva_programacion.php</code>	Formulario de alta; genera <code>ID_Programacion</code> ; carga catálogos
<code>procesar_programacion.php</code>	POST + CSRF → validaciones → <code>INSERT</code> en <code>Programacion</code>
<code>editar_programacion.php</code>	Edición por <code>id_interno</code> (admin)
<code>config/programacion_catalogos.php</code>	Plantas, productos, cuarteo, IDs, generación de código
<code>config/programacion_opl_logistica.php</code>	Relación OPL ↔ conductor/vehículo
<code>config/logisticos_vinculo_maestro.php</code>	Insert en tabla puente <code>logisticos</code>

9.2 Lógica de alta (resumen senior)

Flujo de `procesar_programacion.php`:

1. **Método:** solo POST; si no, redirect a nueva.
2. **CSRF:** `lm_csrf_validar`.
3. **ID de negocio:** valida formato con `programacion_id_programacion_valido` y unicidad en `Programacion.ID_Programacion`.
4. **Normalización de campos:** planta operativa, producto, cliente, OPL, cantidades, fechas, OTIF, etc.
5. **Alta opcional de maestros** (admin): si llega texto “nuevo” (cliente, OPL, solicitante, conductor, vehículo), busca por nombre o inserta ID+nombre mínimo (`programacion_alta_maestro_id_nombre`) con whitelist de tablas/columnas.
6. **Vínculo logístico:** puede registrar relación en `logisticos`.
7. **INSERT** en `Programacion` con estado de pedido/actividad y flags OTIF.
8. **Redirect** a listado con `status=success`.

9.3 Doble representación de planta

Campo	Significado
<code>Planta</code>	ID numérico de maestro (típicamente 1–4)
<code>Planta_Operativa</code>	Código de negocio: <code>BENEFICIO</code> <code>DESPOSTE</code> <code>CELFRIO</code> <code>SUBPRODUCTOS</code>

Productos y tipos de cuarteo se filtran en UI **por planta operativa**; parte del catálogo está **hardcodeado en PHP** además de tablas BD.

9.4 Identificadores

Identificador	Uso
<code>id_interno</code>	PK técnica autoincremental (edición/duplicado)
<code>ID_Programacion</code>	Código de negocio único visible al usuario

Duplicar: `nueva_programacion.php?duplicar_de={id_interno}` (admin).

10. CRUD genérico de maestros

10.1 Hub — `maestros.php`

Punto de entrada de configuración (solo admin). Enlaza a `gestion_tabla.php?tabla=...`

10.2 Motor genérico — `gestion_tabla.php`

Diseño tipo **CRUD meta** (patrón senior reusable):

1. **Whitelist** de tablas permitidas (evita inyección del nombre de tabla).
2. Restricción extra: tabla `user` solo Super Admin.
3. `DESCRIBE $tabla``` para descubrir columnas, PK e ID de negocio.
4. Detección de `id_interno` vs `ID_*` / `Identificacion`.
5. Casos especiales: `empleado` (cédula como PK o `id_interno`), `empleado_programacion` (filtros semana ISO + disponibles).
6. Operaciones crear / editar / eliminar vía formularios + modales JS.
7. Cálculo de siguiente ID de negocio en servidor (`calcularSiguienteIdNegocio`).
8. Al crear OPL, puede vincular a `logisticos`.

Tablas en whitelist

`clientes, corte, departamento, municipio, opl, producto, tipo_de_cuarteo, zona, vehiculo, conductor, solicitante, user, actividad, planta, logisticos, empleado, empleado_descanso, empleado_programacion.`

11. Módulo de personal

Archivo	Responsabilidad
<code>tablero_descansos.php</code>	Tablero semanal (año / semana ISO)
<code>personal_descanso_form.php</code>	Alta/edición/borrado de descansos
<code>personal_programacion_form.php</code>	Alta/edición/borrado de turnos
<code>empleados_importar.php</code>	UI import CSV
<code>config/personal_helpers.php</code>	Disponibilidad, solapes, filtros
<code>config/import_empleados_csv.php</code>	Lógica de importación

Archivo	Responsabilidad
<code>config/schema_empleados.sql</code> (+ alters)	Esquema / migraciones

Entidades:

- `empleado` — PK conceptual `ID_Empleado` (cédula); variantes legacy con `id_interno`.
- `empleado_descanso` — ausencias por año/semana.
- `empleado_programacion` — turnos; actividad, planta, producto.
- `programacion_actividad_extra` — catálogo de actividades adicionales.

La lógica de helpers evita solapes y calcula empleados disponibles para un día/semana dados.

12. Modelo de datos

12.1 Base de datos

- Nombre por defecto: `db_logimeat`
- Charset: `utf8mb4`

No hay dump completo del esquema legado en el repo. La fuente de verdad definitiva en producción es `SHOW CREATE TABLE`. Los SQL en `config/` documentan **extensiones** (personal, usabilidad, planta operativa).

12.2 Tabla central: `Programacion`

Campos conceptuales (según inserts y formularios):

Grupo	Campos
Identidad	<code>id_interno</code> , <code>ID_Programacion</code>
Pedido	<code>Fecha_de_Registro</code> , <code>Solicitante</code> , <code>Medio_de_Comunicacion</code> , <code>Estado</code>
Operación	<code>Cliente</code> , <code>Planta</code> , <code>Planta_Operativa</code> , <code>Actividad</code> , <code>Fecha_de_Operacion</code> (d/m/Y), <code>Hora</code> , <code>Producto</code> ,

Grupo	Campos
	Tipo_de_Cuarteo, Lote, Cantidad (kg), Ciudad, Destino, Ubicacion
Logística	OPL, Conductor, Vehiculo, Observaciones, Telefono
OTIF / calidad	Cantidad_Correcta, Producto_Correcto, Entrega_a_Tiempo, Direccion_Correcta, Pedido_Perfecto, Estado_Actividad

Estados típicos de actividad: PROGRAMADO, EJECUTADO, CANCELADO, etc.

12.3 Maestros (orden de migración sugerido)

Según `scripts/migrar_datos.php`:

1. departamento, municipio, estado, estado_actividad, medio_de_comunicacion, nivel
2. actividad, corte, planta, tipo_de_cuarteo, producto
3. opl, logísticos, solicitante, conductor, vehiculo
4. clientes, user
5. programacion

12.4 Relaciones conceptuales



12.5 Particularidades de modelado

- **FK blandas:** a menudo se guarda ID o nombre; JOINS usan tolerancias (`OR, UPPER(TRIM(...))`).
 - **Case naming inconsistente:** `User` vs `empleado`, `Cientes/clientes` — MySQL en Windows suele ser case-insensitive.
 - **Fechas texto:** implica parseo con `SUBSTRING` / `REGEXP` / conversión a ISO en charts y calendario.
-

13. Flujos de usuario

13.1 Autenticación

None

```
Usuario → login.php → valida User → sesión + rol → index.php
      ↓ fallo
      mensaje error
      ↓ inactividad 15 min
      login.php?error=sesion_expirada
```

13.2 Crear programación

None

```
programacion.php → Nueva → nueva_programacion.php
      → (admin puede crear maestros inline)
      → POST procesar_programacion.php (CSRF)
      → INSERT Programacion
      → programacion.php?status=success
```

13.3 Editar / duplicar

None

```
Admin → editar_programacion.php?id={id_interno}
Admin → nueva_programacion.php?duplicar_de={id_interno}
```

13.4 Mantener maestros

None

```
Admin → maestros.php → gestion_tabla.php?tabla=X
```

→ DESCRIBE + CRUD modal → INSERT/UPDATE/DELETE

13.5 Personal

None

Admin → `tablero_descansos.php` (filtro ISO)
→ `personal_descanso_form` / `personal_programacion_form`
→ `helpers anti-solape` / `disponibles`

13.6 Reset de contraseña

None

`password_reset_request` → token + email SMTP
`password_reset_confirm` → UPDATE User.Clave

14. Seguridad

Control	Estado / ubicación
Sesión con timeout 15 min	<code>auth.php</code>
CSRF en POST críticos	<code>lm_csrf_*</code> + <code>procesar_programacion.php</code> y otros formularios
Whitelist de tablas en CRUD	<code>gestion_tabla.php</code>
Prepared statements PDO	casi todo el acceso a datos
Headers de seguridad	<code>config/seguridad.php</code>
Credenciales fuera de git	<code>conexion.local.php</code>
Contraseñas en claro	deuda técnica — planificar hash
Endpoint calendario sin auth	revisar <code>eventos_calendario.php</code>

15. Frontend y assets

Recurso	Ruta
CSS Tailwind compilado	<code>assets/vendor/tailwind-built.css</code>
Fuente CSS local	<code>assets/vendor/plus-jakarta-local.css</code>
Chart.js	<code>assets/vendor/chart.umd.min.js</code>
FullCalendar	<code>assets/vendor/fullcalendar.index.global.min.js</code>
Logo	<code>assets/colbeef-logo.svg</code>

Principio de diseño: uso sin internet (sin CDN). Regenerar Tailwind según notas en `lm_assets.php` / fuente `tailwind-source.css`.

UI: sidebar fijo oscuro, tipografía Plus Jakarta Sans, componentes con utilidades Tailwind (rounded-2xl, paleta slate/blue).

16. Configuración y despliegue

16.1 Requisitos

- PHP 8.x con PDO MySQL y sesiones
- MySQL/MariaDB `utf8mb4`
- DocumentRoot apuntando al proyecto (o subcarpeta + `web_base`)
- Entorno típico de desarrollo: **Laragon**

16.2 Pasos de instalación

1. Clonar / copiar el repositorio.
2. Crear BD `db_logimeat` e importar dump o migrar con `scripts/migrar_datos.php` + carpeta hermana `logimeat_datos`.
3. Copiar `conexion.local.example.php` → `conexion.local.php` y ajustar credenciales / SMTP / `web_base`.
4. Aplicar schemas parciales si faltan (`scripts/aplicar_schema_*.php`).
5. Verificar acceso a `login.php`.

16.3 Variables / claves locales relevantes

```
PHP
return [
    'host' => '127.0.0.1',
    'db' => 'db_logimeat',
    'user' => 'root',
    'pass' => 'root',
    'port' => 3306,
    'charset' => 'utf8mb4',
    // smtp_* / mail_from / web_base opcionales
];
```

17. Scripts y migraciones

Script	Propósito
scripts/migrar_datos.php	Import masivo desde ../logimeat_datos
scripts/aplicar_schema_*.php	Aplicar SQL de usabilidad / programación operacional
scripts/aplicar_migration_planta_operativa.php	Columna/código Planta_Operativa
scripts/aplicar_migrar_empleado_pk.php	Migración PK empleado (cédula)
scripts/import_empleados_ejecutar.php	CLI import empleados
scripts/import_descansos_dashboard.php	Import descansos
scripts/test_mail_smtp.php	Prueba SMTP

SQL de referencia en config/schema_*.sql y config/alter_*.sql.

18. Convenciones para un ingeniero senior

1. **No introducir Composer/MVC** sin acuerdo de arquitectura: el sistema es page-controller a propósito.
 2. **Toda pantalla privada** debe `require_once 'auth.php'` al inicio.
 3. **SQL dinámico de nombres de tabla/columna** solo con whitelist (como `gestion_tabla` / alta maestros).
 4. **Fechas de operación:** asumir `d/m/Y` string; convertir explícitamente a ISO para ordenar/graficar.
 5. **Planta:** mantener sincronía entre ID numérico y `Planta_Operativa`.
 6. **Helpers nuevos:** prefijo `lm_` o dominio (`programacion_`, `personal_`).
 7. **CSRF** en todo POST que mute estado.
 8. **No hardcodear secretos** en archivos versionados; usar `conexion.local.php`.
 9. **Tolerancia a legado:** muchos `try/catch (Throwable)` alrededor de maestros opcionales — no romper pantallas si falta una tabla.
 10. **Assets:** no apuntar a CDN; regenerar Tailwind localmente.
 11. **Roles:** usar `lm_es_admin()` / `lm_es_super_admin()`, no comparar strings ad hoc en cada página.
 12. **Documentar cambios de esquema** con SQL en `config/` + script `aplicar_*` cuando sea posible.
-

19. Mapa de archivos por responsabilidad

Autenticación y cuenta

Archivo	Rol
<code>login.php</code>	Login / logout
<code>auth.php</code>	Gate sesión, roles, sidebar
<code>cambiar_password.php</code>	Cambio de clave autenticado
<code>password_reset_request.php</code> / <code>password_reset_confirm.php</code>	Flujo forgot password
<code>config/password_reset_helpers.php</code>	Tokens / tabla reset
<code>config/seguridad.php</code>	Cookies, headers, CSRF

Operación logística

Archivo	Rol
programacion.php	Listado
nueva_programacion.php	Alta UI
procesar_programacion.php	Alta backend
editar_programacion.php	Edición
logistica.php	Conductores / vehículos
view_data.php / eventos_calendario.php	Calendario

Analítica

Archivo	Rol
index.php	Dashboard
estadisticas.php	Estadísticas
otif.php	Calidad OTIF

Maestros / personal / usabilidad

Archivo	Rol
maestros.php	Hub configuración
gestion_tabla.php	CRUD genérico
tablero_descansos.php	Tablero personal
personal_*_form.php	Formularios personal
empleados_importar.php	Import CSV UI
tablero_usabilidad.php	Métricas de uso

20. Guía de onboarding

Si necesita...	Empiece por...
Entender el negocio	Tabla <code>Programacion</code> + <code>nueva_programacion.php</code> / <code>procesar_programacion.php</code>
UI, menú y roles	<code>auth.php</code> , <code>login.php</code>
Maestros	<code>maestros.php</code> → <code>gestion_tabla.php</code>
Personal	<code>config/schema_empleados.sql</code> , <code>tablero_descansos.php</code>
Desplegar	<code>conexion.local.example.php</code> → Laragon + MySQL
Datos iniciales	<code>scripts/migrar_datos.php</code> + <code>logimeat_datos</code>
Ver el flujo completo	Abrir <code>diagrama_flujo_logimeat.drawio</code> en diagrams.net

21. Diagrama de flujo

El diagrama editable de **todo el flujo del software** está en:

[diagrama_flujo_logimeat.drawio](#)

Compatible con **draw.io** / **diagrams.net** (abrir el archivo directamente en la aplicación de escritorio o en <https://app.diagrams.net/>).

Incluye:

- Autenticación y roles
- Navegación por módulos
- Flujo de programación (alta / edición / duplicado)
- Maestros y CRUD genérico
- Personal

- Dashboard / estadísticas / OTIF / calendario
- Persistencia MySQL

22. Glosario

Término	Definición
OPL	Operador / entidad logística asociada a conductor y vehículo
OTIF	<i>On Time In Full</i> — calidad de entrega (tiempo, cantidad, producto, dirección)
Planta operativa	Código de negocio: BENEFICIO, DESPOSTE, CELFRIO, SUBPRODUCTOS
ID_Programacion	Identificador de negocio único de un viaje/movimiento
id_interno	Clave técnica interna (PK)
WORKBEEF	Portal intranet Colbeef al que LogiMeat enlaza
Page controller	Un archivo PHP = una pantalla/endpoint
Whitelist de tablas	Lista fija de tablas CRUD permitidas
FK blanda	Relación lógica sin FK estricta en BD

Anexo A — Checklist de revisión de código

- ☐ ¿La página privada incluye `auth.php`?
- ☐ ¿Los POST mutantes validan CSRF?
- ☐ ¿Los nombres de tabla/columna dinámicos están en whitelist?
- ☐ ¿Se usan prepared statements para valores?
- ☐ ¿Se escapa salida HTML con `htmlspecialchars(..., ENT_QUOTES, 'UTF-8')`?
- ☐ ¿Los nuevos maestros respetan generación de ID de negocio?
- ☐ ¿Las fechas de operación se manejan como `d/m/Y` de forma consciente?

☐ ¿Se evitó introducir dependencias CDN?

Anexo B — Deuda técnica conocida

1. Contraseñas en texto plano en `User.Clave`.
 2. Posible endpoint de calendario sin gate de autenticación.
 3. OTIF parcialmente hardcodeado en pantalla de calidad.
 4. Ausencia de schema SQL completo versionado de tablas legacy.
 5. Naming inconsistente de tablas/columnas (legado).
 6. Sin suite de tests automatizados.
-

Fin del documento. Mantener sincronizado este Markdown y el archivo `.drawio` ante cambios estructurales del sistema.